WPT Day 2

Thu 22 Oct 26 (Evening) · CSS selectors and specificity, internal / external / inline CSS, font & text properties, the `div` tag, borders, padding, margin, the box model, and colour — explained from first principles

---

Table of Contents

1. CSS Selectors: Choosing Exactly What to Style
2. Selector Specificity: When Rules Collide
3. Three Places to Write CSS: Internal, External and Inline
4. Centering Images with CSS, Not HTML
5. Font and Text Properties
6. The div Tag: Dividing a Page into Sections
7. Border Properties and Rounded Corners
8. Padding, Margin and the Box Model
9. How CSS Represents Colour
10. Combined Worked Example: A Styled Profile Card Page
11. Quick Revision Checklist
12. Practice Exercises
13. Practice Answers

CSS Selectors: Choosing Exactly What to Style

The problem a selector solves

On Day 1 you met the whole of CSS syntax in one shape:

```
selector { property: value; }
```

Day 2 is mostly about the first word of that shape. Why does CSS even need a "selector"? Reason it from the page itself. A real page contains hundreds — the lecture says 1,000+ — of elements: headings, paragraphs, spans, images, divisions. When you write `color: red`, you have answered *what change* but not *where it applies*. Should one heading turn red, or every heading, or three unrelated elements scattered across the page that happen to share a trait?

The **selector is the addressing system of CSS**: it names *which* elements a rule applies to. The property/value pairs inside the braces are the instruction; the selector is the address it is delivered to. Every selector type in this chapter is a different kind of address, suited to a different targeting problem:

Selector	Address it uses	Reaches
Tag selector	The element's tag name	Every element of that kind
ID selector	One element's unique `id`	Exactly one element
Class selector	A shared `class` name	Any chosen group of elements
Universal selector	`*`	Every element on the page

Learn them as answers to four questions: *all of this kind?* (tag), *that one specific element?* (ID), *this hand-picked group?* (class), *literally everything?* (universal). The rest of the chapter is detail.

Tag selector: style every element of a kind

Mechanism. Write the tag name directly as the selector — no angle brackets, no prefix symbol:

```
h1 {
    color: red;
}
```

This rule applies red to **every `<h1>` on the page**. The lecture states the consequence at scale so you feel it: **if 100 `<h1>` tags exist, all 100 receive the same CSS.** There is no "first one only" or "just this one" with a tag selector; the address is the *kind*, so every member of the kind is reached.

When to use it — exactly the situation it describes: when *all* matching tags should share identical styling. All headings in the site's brand colour, all paragraphs at the same base size. The moment one heading needs to be different from the other ninety-nine, the tag selector has reached its limit and you need a narrower address.

> **Pitfall:** The tag selector is the tag name *without* angle brackets: `h1`, never `<h1>`. Day 1's rule has not changed — an angle-bracketed "selector" is invalid CSS and is silently ignored.

ID selector: one element, uniquely identified

The problem. Sometimes exactly *one* element must be singled out — one specific heading, one specific box — while other elements of the same tag stay untouched. A tag selector cannot do this; it cannot tell two `<h1>`s apart.

Mechanism, in two halves:

1. In the HTML, add an `id` attribute to the element:

```
<h1 id="head1">Welcome</h1>
```

2. In the CSS, target that ID with a **hash prefix**:

```
#head1 {
    color: red;
}
```

Three rules govern IDs, and all three are examinable:

- **An ID can be added to any HTML tag** — it is not exclusive to headings. Any element that needs a unique name can carry one.
- **Every ID must be unique across the entire page.** `id="head1"` may appear once. This uniqueness is not decoration; it is what makes the selector's promise ("exactly one element") true.
- **The `#` is mandatory.** Writing the name without the hash does **not** target the ID, as the next rule shows:

```
head1 {
    color: red;
}
```

CSS reads a bare word as a **tag selector by default**, so this rule goes looking for `<head1>` elements — which do not exist — and quietly does nothing. The `#` is what switches the word from "tag named head1" to "the element whose ID is head1".

What happens if you duplicate an ID anyway? HTML will not stop you — the page still renders, no error stops execution. But duplicate IDs trigger **browser console warnings/errors**, and the uniqueness every tool (CSS, JavaScript, labels) relies on is broken underneath you. The lecture's line on this deserves to be quoted verbatim, because it describes all of HTML/CSS behaviour in one sentence:

> *"HTML is an error-free language — you won't get errors, but bugs will follow you."*

Background: "Error-free" here does not mean the browser approves of your code; it means the browser's design philosophy is to *render something no matter what* rather than refuse. A compiler in other modules rejects bad code loudly. HTML and CSS instead guess, skip, or tolerate — which is why Day 1 sent you to DevTools for crossed-out rules, and why silent, following bugs are this technology's characteristic failure mode.

> **Exam:** ID selector = `#name { … }`, targets **one** element, ID must be **unique on the page**, and the `id` attribute may be placed on **any** tag. Omitting `#` turns the rule into a tag selector that matches nothing.

Class selector: a shared trait across many elements

The problem. Now the opposite situation from ID: several elements — possibly of *different tags* — should share the same styling, but *not every* element of those tags should. The lecture's exact example: the **second `<h1>`, the first `<p>`, and the third `<span>`** should all be blue, while all the other headings, paragraphs and spans are not. Tag selectors are too broad (they would take every `h1`, `p` and `span`); ID selectors are too singular (one per element, and each would need its own duplicated rule).

Mechanism, again in two halves:

1. In the HTML, give each chosen element the same `class` attribute value:

```
<h1 class="blue-font">Second heading</h1>
<p class="blue-font">First paragraph</p>
<span class="blue-font">Third span</span>
```

2. In the CSS, target the class with a **dot prefix**:

```
.blue-font {
    color: blue;
}
```

One CSS rule now covers all three elements. The defining contrasts with ID:

- **The same class value can be reused on any number of elements** — that is its entire purpose. Uniqueness is ID's rule; sharing is class's rule.
- **It eliminates duplicate CSS blocks.** The alternative — a separate ID rule per element, each repeating `color: blue;` — is the same instruction copied three times. The class says it once and attaches it three times. When the colour later changes, the class version is edited in one place.

Notice what the class *means*: not "this element" (ID) and not "this kind of element" (tag), but "**elements I have marked as sharing this trait**". You invent the trait; the browser just follows the name.

> **Exam:** Class selector = `.name { … }` (dot prefix required), reusable on **many** elements, and its payoff is **one rule instead of duplicated blocks**. The lecture's example group — second `h1`, first `p`, third `span`, all `class="blue-font"` — is the example to reproduce.

> **Pitfall:** Confusing the two prefixes. `#` = ID = one element. `.` = class = many elements. Writing `.head1` when `head1` is an ID (or `#blue-font` when it is a class) produces a rule that matches nothing and fails silently.

Universal selector: everything, at once

Mechanism. A single asterisk targets **every element on the page**:

```
* {
    font-size: 16px;
}
```

When it is practical: when a single CSS property must apply to **all 1,000+ tags** on a page — the lecture's example is a base `font-size`. Writing that per-tag would mean a rule for every tag type; `*` says it once for everything that exists.

A deliberate recommendation from the lecture: prefer the universal selector over applying such page-wide CSS **directly to** `body`, because styling `body` can **unintentionally affect backgrounds**. The reasoning underneath, made explicit: `body` styling behaves specially — the body's background in particular propagates to the whole canvas/page in ways that surprise beginners. A universal rule states plainly "every element gets this property" without entangling the page background. When you want *everything*, address *everything*.

> **Exam:** Universal selector is `* { … }`, applies to **every element**, and is the lecture's recommended way to apply a property page-wide — preferred over styling `body`, which can unintentionally affect backgrounds.

The four selectors, side by side

Selector	Syntax	Prefix	Cardinality	Choose it when
Tag	`h1 { }`	none	All elements of that tag	Every element of a kind shares one style
ID	`#head1 { }`	`#`	Exactly **one** element (unique per page)	One specific element must differ from all others

Selector	Syntax	Prefix	Cardinality	Choose it when
Class	`.blue-font { }`	`.`	**Many** elements, any tags	A hand-picked group shares a trait
Universal	`* { }`	— (the `*` itself)	**Every** element	One property must apply page-wide

---

Selector Specificity: When Rules Collide

The problem: one element, several rules, one winner

Selectors overlap by design. Nothing stops this from happening:

```
*         { color: blue; }
h1        { color: green; }
.blue-font { color: red; }
#head1    { color: yellow; }
```

```
<h1 id="head1" class="blue-font">Which colour am I?</h1>
```

This one `<h1>` is simultaneously an `h1` (tag rule), a carrier of `class="blue-font"` (class rule), the owner of `id="head1"` (ID rule), and an element (universal rule). Four rules set `color` to four different values, but the heading can only *be* one colour. CSS needs a deterministic tie-breaker — one that does not depend on luck or file order. That tie-breaker is **specificity**.

The specificity order

When multiple selectors target the same element, **specificity determines which rule wins**, in this fixed order, highest to lowest:

```
ID  >  Class  >  Tag  >  Universal
(strongest)                (weakest)
```

Read `>` as "beats". An ID rule beats a class rule; a class rule beats a tag rule; a tag rule beats the universal rule. The logic is intuitive once seen as *narrowness of address*: the more precisely a rule identifies its target, the more it presumably reflects a deliberate intention for *that* element, so it wins. `*` addresses a thousand elements with one breath; `#head1` addresses one element by name. The deliberate, specific instruction overrides the broadcast.

The lecture's live demonstration

This was demonstrated live in the session, with exactly the setup above:

- Universal selector gave **blue**
- Tag selector gave **green**
- Class selector gave **red**
- ID selector gave **yellow**

Result: the heading rendered yellow — the ID won, because ID sits at the top of the specificity order. If you remove the ID rule, the same element falls to red (class beats tag and universal); remove the class rule too and it falls to green; remove the tag rule and only blue remains. Worth doing once with your own hands: comment rules out one at a time in DevTools and watch the colour step down the ladder.

Only conflicting properties are overridden

This is the refinement students most often miss, and the lecture states it explicitly. Specificity is a **per-property** contest, not a winner-takes-the-whole-rule contest:

- If the universal selector sets `font-style` and the ID selector sets `color`, **both are applied** — they do not conflict, so there is nothing for specificity to decide.
- Only when both define the *same* property (both define `color`) does the higher-specificity rule's value override the lower's.

So a losing rule is not deleted or ignored; it still contributes every property that no stronger rule has claimed. A page's final appearance for one element is typically a *merge*: base properties arriving from broad, weak selectors, and the few contested properties settled by the strongest selector that mentions them.

```
*        { font-style: italic;  color: blue;   }
#head1   {                      color: yellow; }

Result on #head1:  font-style: italic  (from *, uncontested)
                   color: yellow       (from #head1, won the contest)
```

> **Exam:** Specificity order is **ID > Class > Tag > Universal**, and it resolves **only conflicting properties** — non-conflicting properties from lower-specificity rules still apply. The live result to quote: blue / green / red / **yellow — ID won**.

> **Pitfall:** Believing a stronger selector "cancels" the weaker rule entirely. It cancels nothing; it only wins the properties both sides actually set. Half of all "my CSS disappeared" confusion is this misunderstanding in one direction or the other.

Three Places to Write CSS: Internal, External and Inline

The problem: where should the rules live?

Day 1 put CSS in a `<style>` tag inside `<head>` and moved on. But that was one choice among three, and the choice has consequences that only appear when a site grows past one page: fifty pages each carrying a copy of the same rules means fifty edits for one colour change. Day 2 names all three placements — **types of CSS** — and ranks them.

Internal CSS: `<style>` in the same file's `<head>`

CSS written **inside a `<style>` tag, placed in `<head>` of the same HTML file**:

```
<head>
    <style>
        h1 { color: green; }
    </style>
</head>
```

Everything in the Day 1 session and the morning of Day 2 used internal CSS. It is suitable for **single-page styling** — quick demos, one-off pages, learning. Its structural weakness appears only with scale: with multiple HTML files, **each file repeats the same rules**, and a site-wide change must be repeated file by file.

External CSS: a dedicated `.css` file, linked in

CSS written in its **own file** — conventionally `style.css` — containing selectors and rules directly:

```
/* style.css — no <style> tag in here, just rules */
h1 {
    color: green;
}
```

Two facts about the `.css` file itself, both easy to fumble:

- **No `<style>` tag is needed (or allowed) inside the `.css` file.** The file *is* CSS; you write selectors and rules directly, top to bottom.

- **The file does nothing until it is explicitly linked.** Creating `style.css` next to your HTML changes nothing on the page; the browser must be told the file exists and belongs to this page. Unlinked CSS is simply never applied.

The link. Inside `<head>` of *each* HTML file that wants the styles:

```
<link rel="stylesheet" type="text/css" href="style.css">
```

The `<link>` tag carries **three attributes**, and the lecture explains each:

Attribute	Value	Meaning
`rel`	`"stylesheet"`	A **predefined** value; declares the linked file's **relationship to the HTML** — "this file is my stylesheet"
`type`	`"text/css"`	Specifies the **content type** of the linked file. `text` = the file is textual, **not binary** (not `.mp3` / `.mp4`). The lecture notes this may be skipped, but only at the cost of **backward compatibility** — older browsers expect it
`href`	`"style.css"`	The **path to the CSS file**, following the **same relative-path rules as linking images**

Why bother? Two advantages, and they are the reason real projects default to external CSS:

1. **Modularity** — HTML and CSS are separated into distinct files. Structure lives in one place, presentation in another; each file has one job (the Day 1 division of labour, now enforced by the filesystem).
2. **Reusability** — **one external file can be linked to 50+ HTML pages**. Change a rule once in `style.css` and **a single edit updates all of them**. Compare internal CSS, where the same change is fifty separate edits, forty-nine of which someone will forget.

Background: "Relative path" in `href` means the path is interpreted *relative to the HTML file's own location*: `href="style.css"` means "a file named `style.css` in the same folder as this HTML file". Put the CSS in a subfolder and the path must say so (e.g. `css/style.css`), exactly as with image `src` paths. Most "my external CSS isn't working" reports are, in the end, a path that does not point where the author thinks it points — check `href` first, then check that the link tag is present at all.

Inline CSS: the `style` attribute on the tag itself

CSS written **directly inside an individual tag** using the `style` attribute:

```
<p style="color: red;">This paragraph alone is red.</p>
```

No selector is needed — the rule is physically attached to the one element it styles. The lecture's analogy for its precision is quoted verbatim, in the instructor's words:

> *"Inline is like walking into someone's house to shoot — most accurate, highest preference."*

Unpack the analogy: no address ambiguity, no intermediary — the instruction is delivered *inside* the target itself, so nothing can outrank it (next section). But the same quality makes inline CSS the **least maintainable at scale**: every element carries its own private copy of its styling, there is no single place to edit, and reusability is zero. A site-wide colour change in inline CSS is a search through every tag of every page. Highest power, worst organisation.

Preference order when all three conflict

Inline has an exact counterpart to selector specificity. When the **same element** is styled by all three placements and the rules conflict, the preference order is:

```
Inline  >  Internal  >  External
(strongest)                (weakest)
```

So an inline `style="color: red;"` beats an internal `<style>` rule, which beats a rule in the linked `.css` file — for that element. And the same refinement as specificity applies: **only conflicting properties are overridden**; non-conflicting properties from the lower-preference sources still apply. An external rule can still supply the `font-size` of an element whose `color` was claimed inline.

Why does this order make sense? It is the specificity idea again, expressed by *distance*: the closer the rule physically sits to the element, the narrower and more deliberate its aim — the attribute is *on* the element, the `<style>` block is in the *same document*, the `.css` file is *another file entirely, shared with 50 pages*. Proximity wins conflicts; sharing wins maintenance. Professional practice therefore defaults to external CSS and reaches for inline only deliberately — the preference order is a conflict-resolution rule, not a recommendation to write everything inline.

> **Exam:** Three types — **internal** (`<style>` in `<head>`, same file), **external** (separate `.css` file, linked with `<link rel="stylesheet" type="text/css" href="…">` inside `<head>`), **inline** (`style` attribute on the tag). Preference on conflict: **Inline > Internal > External**. External's advantages: **modularity and reusability** (one file, 50+ pages, one edit).

> **Pitfall:** Two classic external-CSS failures: (1) writing a `<style>` tag *inside* the `.css` file, and (2) creating `style.css` but never linking it — the file is inert until `<link>` connects it to the page.

Centering Images with CSS, Not HTML

The problem raised at the start of the session

The evening session opened with a leftover problem: **centering an image using HTML's `align` attribute did not work as expected.** This small failure carries a general lesson about where presentation belongs, so the lecture answers it as a rule rather than a trick.

The recommendation

- **Do not use the HTML `align` attribute for centering** images (or, by extension, other elements).
- The HTML `<center>` **tag is deprecated in HTML5** — it exists in old pages, browsers still tolerate it, but it is officially retired and must not be used in new code.
- **The correct approach is to use CSS** for alignment and centering of images and other elements.

This is the Day 1 division of labour enforcing itself again. `align` and `<center>` were HTML trying to do presentation — styling smuggled into structure, like the table `border` / `cellpadding` attributes from Day 1. HTML5's direction is consistent: structure in HTML, *all* presentation — including where things sit on the page — in CSS. When an HTML styling attribute "doesn't work", the lecture's message is: it was never the right layer to ask.

Background (the concrete CSS, which the lecture pointed to without spelling out): an `<img>` is an inline element by default, so the two standard CSS ways to centre it are (a) centre it as text — put `text-align: center;` on its **parent**; or (b) make it a block and centre the block — `display: block; margin-left: auto; margin-right: auto;` on the image itself. Both are "use CSS for centering"; which one you pick depends on context, and margin mechanics arrive later in this chapter. What neither method uses is `align=` or `<center>`.

> **Exam:** Centering is a **CSS** job. The `align` attribute is not to be used for it, and `<center>` is **deprecated in HTML5**.

Font and Text Properties

The problem: text is not one knob, it is seven

Until now "styling text" has meant little more than `color`. But look at any professionally typeset page: letters have a size, a typeface, a weight, a slant, a case, possibly a line struck through them. Each of those is an independent visual fact, and CSS gives each its own property. This chapter is the catalogue — seven properties, each answering one question about text.

The font properties

Property	Controls	Lecture's example / note
`font-size`	Size of the text, in pixels	`font-size: 24px;` visibly increases paragraph text
`font-family`	The typeface, by name	`Segoe UI Light`, `Arial`, `Times New Roman`, `Algerian`
`font-weight`	Boldness, numerically	`900` = heavy bold
`font-style`	Slant of the text	`italic` or `oblique`
`font-variant`	Small-caps effect	`small-caps` (see below)

Unfold the ones with conditions attached:

`font-family` depends on the user's machine. The value names a font — Segoe UI Light, Arial, Times New Roman, Algerian were the session's examples — but naming is requesting, not shipping: **the font must exist on the user's system to render correctly.** Declare `font-family: Algerian;` to a visitor whose machine lacks Algerian and their browser silently substitutes a default. Your CSS travels; the font files (mostly) do not. This is why real stylesheets conventionally list fallbacks — but even the single-name form taught here obeys the rule: no font on the system, no rendering of that font.

`font-weight` is numeric, and the font sets the limits. `font-weight: 900;` asks for heavy bold. But the **actual visual result depends on how much weight variation the chosen font family supports** — a family that ships only regular and bold can offer two real steps no matter what number between 100 and 900 you request; a family with a full weight range renders the gradations. The number is a request into the family's available range, not a guarantee of nine visually distinct bolds.

`font-variant: small-caps` does something subtler than uppercase. With `font-variant: small-caps;`, **all letters appear as capitals, but the first letter of each word renders at a larger size than the rest** — small capitals following a full-size capital, the classic book-heading look. The

lecture adds the honest practitioner's note: this is **rarely used in practice**. Learn it for recognition and exams; expect to meet it seldom in production code.

```
p {
    font-size: 24px;
    font-family: Arial;
    font-weight: 900;
    font-style: italic;
    font-variant: small-caps;
}
```

The text properties

Where the `font-*` properties describe the letters themselves, the `text-*` properties describe transformations and decorations applied to the text as a whole.

`text-transform` **— convert casing without changing the HTML content.** The stored/typed text stays exactly as written; only its *displayed* case changes:

Value	Effect
`uppercase`	ENTIRE TEXT IN CAPITALS
`lowercase`	entire text in lowercase
`capitalize`	First Letter Of Every Word Becomes Uppercase

Why transform in CSS rather than just typing capitals in the HTML? Because case here is *presentation*, and presentation is CSS's department: the same underlying text (say, a username from a database) can be displayed SHOUTED on one page and Capitalized on another without touching the data. Change your mind about the design and you edit one property instead of retyping content.

`text-decoration` **— add lines to text:**

Value	Effect	Typical reading
`underline`	Line **below** the text	Emphasis; the classic link look
`overline`	Line **above** the text	Rare, decorative
`line-through`	Line **through** the middle	Marks text as cut/deleted — a struck price, a completed task

The decoration is a line whose *position* is the whole meaning: below, above, or through. `line-through` carries semantics users read instantly — ~~old price~~ — which is why it is described as

marking text as cut/deleted rather than merely "a line in the middle".

> **Exam:** `font-size` (px), `font-family` (must exist on the **user's system**), `font-weight` (numeric, e.g. `900`; result limited by the family's supported weights), `font-style` (`italic` / `oblique`), `font-variant: small-caps` (all capitals, first letter larger; rarely used), `text-transform` (`uppercase` / `lowercase` / `capitalize` — display only, HTML unchanged), `text-decoration` (`underline` / `overline` / `line-through`).

The div Tag: Dividing a Page into Sections

The problem: a page needs rooms, not one open hall

So far your pages are flat sequences: a heading, a paragraph, a form, in one stream. A real page is not a stream — it has a **navbar** across the top, a **footer** at the bottom, perhaps an **image slider**, a **side navigation bar**. Each of those is a *region*: a group of related elements that must be sized, coloured, and positioned *as a unit*. A stream of individual tags gives you no way to say "these five elements are one thing — treat them together".

`<div>` is that grouping tool. The name itself is the definition: **`div` stands for "division"**, and its primary purpose is **to break a web page into distinct sections** — navbar, footer, image slider, side navigation bar, and every other region a layout needs. A **`div` acts as a container**: you group related elements (images, links, buttons) inside one `div`, and thereafter the container can be styled and moved as one box.

The lecture's forecast is worth keeping verbatim: "*in real-time applications you will always use div tags a lot.*" From here on, `div` is the structural unit of nearly every page you build.

```
<div>
    <h3>Special Offer</h3>
    <p>Grouped heading, text and button – one section:</p>
    <button type="button">Shop Now</button>
</div>
```

Default behaviour: an unstyled div is almost nothing

Before styling a `div`, know what it is *without* CSS — the lecture demonstrated this precisely:

Aspect	Default (no CSS applied)
Width	Inherits **100% of the parent/browser width** (driven by the block-level content inside)

Aspect	Default (no CSS applied)
Height	**Determined entirely by the content inside** — no fixed height by default
Background	**Inherits the body background** (white by default); no independent background colour

This was **demonstrated live via DevTools hover**: with one paragraph inside, the `div` showed **752px width** (the full available width) and roughly **18.8px height** — exactly one line of text tall. An `<h3>` was then added inside, and the height **grew to roughly 59px**. Nothing about the `div` was edited; the height simply *follows the content*. That is the default contract: full width, content-height, invisible background. A `div` is a section-shaped emptiness until CSS gives it dimensions and colour.

What CSS changes

With CSS, **every default can be overridden**:

```
div {
    width: 300px;
    height: 300px;
    background-color: green;
    color: white;
}
```

- `width: 300px; height: 300px;` — explicit dimensions replace "100% wide, content tall" with a fixed 300×300 box.
- `background-color: green;` — the box gets a background of its own instead of inheriting the body's white.
- `color: white;` — changes the font colour of **all child elements inside the div**: style the container once and its contents follow.

That last point is the container idea paying rent — one rule on the parent recolours every heading, paragraph and button inside it.

Many divs, told apart by ID

A real page has many `div`s — navbar, slider, footer — each needing *different* styling. A tag selector `div { }` would style them all identically (the tag selector's reach, from Chapter 1, now working against you). So: **use the ID attribute to distinguish and individually style multiple divs on the same page** — `#navbar { }`, `#footer { }`, `#slider { }`, each unique, each targeted alone. This is the ID selector's intended use arriving in practice: one element, one name, one rule.

> **Exam:** `div` = **division**; a **container** that breaks the page into sections (navbar, footer, image slider, side navigation). Defaults: **width 100% of parent, height set by content, background inherited from body**. CSS overrides all three (`width` / `height` , `background-color` , `color` for children). Multiple divs are distinguished by **ID**.

Border Properties and Rounded Corners

The problem: a box needs an outline — and one property is not enough

The 300×300 green `div` from the previous chapter is a filled square floating on the page. To finish it as a visible *box*, it needs an outline. Beginners' first instinct is a single `border: black;` — and nothing appears. The lecture's central fact explains why: **three values are required to render a visible border.** A border is not one decision but three independent ones — how thick, what colour, and drawn in what pattern of line — and CSS makes you supply all three.

The three required values

Property	Decision it makes	Lecture's example
`border-width`	**Thickness** of the border line	`10px`
`border-color`	**Colour** of the border	`black`
`border-style`	**Visual style** of the line	see below

The `border-style` values demonstrated:

Style	Appearance
`solid`	Plain continuous line
`dashed`	Broken dashes
`dotted`	Series of dots
`double`	Two parallel lines

Why is `border-style` the one that silently kills borders? Because thickness and colour describe a line that has not yet been given an *existence*: until a style says drawn-how, there is no line to be thick or coloured. Omit `border-style` and the other two values describe nothing — the characteristic pitfall of this chapter, and it fails silently in the usual HTML/CSS way.

Rounded corners: `border-radius`

Once a border (and a box) exists, its corners are square by default. **`border-radius` rounds the corners of the element**: `border-radius: 20px;` produces a **visibly curved edge**. The value is, in effect, how much corner to cut away and replace with curve — small values soften the corner slightly; larger values curve it more.

The lecture's demonstration

All four properties were demonstrated live on the 300×300 green `div` from the previous chapter — the same box accumulating its final form:

```
div {
    width: 300px;
    height: 300px;
    background-color: green;
    color: white;
    border-width: 10px;
    border-color: black;
    border-style: solid;    /* try dashed, dotted, double */
    border-radius: 20px;
}
```

One element now exhibits the whole chapter: fixed size, own background, inherited-by-children text colour, a 10px black border in a chosen pattern, and corners rounded by 20px. Keep this listing; the next chapter adds space inside and outside it.

> **Exam:** A visible border needs **three** values — `border-width` (thickness), `border-color`, `border-style` (`solid` / `dashed` / `dotted` / `double`). `border-radius` (e.g. `20px`) rounds corners. All were demonstrated on the 300×300 green `div` .

> **Pitfall:** Setting `border-width` and `border-color` and seeing nothing. Without `border-style` there is no line to thicken or colour.

Padding, Margin and the Box Model

The problem: boxes need breathing room — in two different places

The bordered green box exists, but two distinct discomforts remain. Text inside it touches the border. And the box itself touches whatever sits next to it. These feel like one problem ("not

enough space") but they are two: space is needed **inside** the box, between its border and its content, *and* **outside** the box, between it and its neighbours. CSS keeps them rigorously separate, with two properties that beginners swap constantly:

- **Padding** — **internal** spacing: space between the element's **border and its content**.
- **Margin** — **external** spacing: space between the element's **outer edge and surrounding elements**.

Inside versus outside. Everything else in this chapter is notation for those two ideas.

Padding: space inside the box

Padding pushes the **content** away from the **inner edge** of the border; **the `div` itself does not move** — its content retreats within it.

Directional properties name each side separately:

```
div {
    padding-left: 20px;
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 0px;
}
```

`padding-left: 20px;` pushes content **20px away from the left inner edge** — the box stays put, the text shifts right.

Shorthand: several sides in one line. Writing four directional lines every time is verbose, so `padding` accepts multiple values. The whole skill is knowing which value lands on which side:

Shorthand form	Meaning	Example
4 values	Read **clockwise: top, right, bottom, left**	`padding: 20px 10px 15px 40px;` → top 20, right 10, bottom 15, left 40
2 values	First = **top and bottom**, second = **left and right**	`padding: 10px 20px;` → top/bottom 10, left/right 20
1 value	**All four sides equal**	`padding: 10px;` → every side 10

The 4-value order — **top, right, bottom, left, clockwise from the top** — is the one to memorise cold; the 2-value and 1-value forms are just its symmetric compressions. (Reason about the 2-value form rather than memorising it separately: if left and right must match and top and bottom must match, two numbers suffice — vertical first, horizontal second.)

The hidden cost of padding: the box grows. By default, **adding padding increases the element's total rendered dimensions**. A `width: 300px` box with 20px padding on both sides renders 340px wide: the declared width covered the *content*, and padding is added *outside* that measure. The lecture demonstrated the growth live: **without `box-sizing`, a 300×300 `div` grew to 359×329 after padding was added.**

The fix: `box-sizing: border-box`. Add this property and the arithmetic inverts: **the element stays at its declared width/height, and padding is absorbed within it** — the content area shrinks to make room for the padding instead of the box expanding to make room for both. Demonstrated on the same div: **with `border-box` it stayed at 300×300**.

```
div {
    width: 300px;
    height: 300px;
    box-sizing: border-box;  /* 300px now INCLUDES padding + border */
    padding: 20px;
}
```

Background: with `border-box`, the declared `width` / `height` measure the **border box** — content + padding + border together — which is why padding (and border) must be absorbed *inward*. Without it (the default, called `content-box`), declared dimensions measure the content alone and padding/border stack *outward*. `border-box` is the near-universal professional default precisely because "a 300px box is 300px" is the arithmetic layout intuition expects.

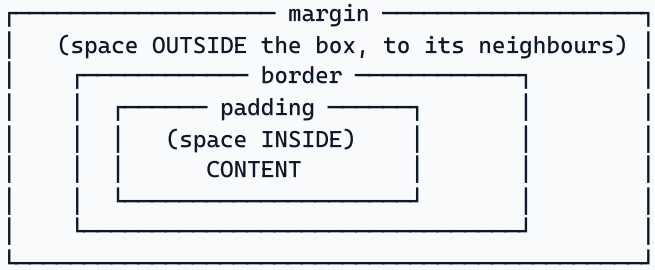
Margin: space outside the box

Margin is padding's mirror: **external spacing between the element's outer edge and surrounding elements.** It does not move content within the box; it moves (the space around) the **whole box**:

```
div {
    margin-left: 50px;
}
```

`margin-left: 50px;` **shifts the entire div 50px away from the left edge of the page**. Directional properties mirror padding's — `margin-left`, `margin-top`, `margin-right`, `margin-bottom` — and **the same shorthand variations apply** (4 values clockwise, 2 values vertical/horizontal, 1 value all sides).

The one-line test that keeps the two straight forever: *padding moves the content away from the border; margin moves the box away from its neighbours.*



The box model: seeing all of it in DevTools

Notice the diagram above *is* a model of every element: content at the centre, wrapped in padding, wrapped in border, wrapped in margin. This is the **CSS box model**, and you do not have to imagine it — the **browser's DevTools inspect panel displays it as a visual diagram** for any selected element, showing the content dimensions, padding, border thickness, and margin currently in effect.

The lecture's shown example, worth reading once value by value: **content 240×270px**, `padding-left: 20`, `padding-top: 10`, `padding-right: 20`, `padding-bottom: 0`, **border ≈ 9.6px**, **margin 0**. That single diagram explains the element's entire spatial footprint — which is its purpose: it is **useful for debugging layout issues without guessing values**. When a box is "somehow too wide", the answer (content? padding? border? margin?) is written in the box-model panel, the way the answer to "why is this rule ignored" is written in crossed-out Styles. DevTools remains the browser telling you what it actually did.

> **Exam: Padding = inside** (border ↔ content); **margin = outside** (box ↔ neighbours). Shorthand: 4 values = **top, right, bottom, left (clockwise)**; 2 values = top/bottom, left/right; 1 value = all sides. Padding **grows** the box by default; `box-sizing: border-box` keeps it at its declared size (lecture demo: 359×329 growth without, 300×300 with). The **box model** (content → padding → border → margin) is visible in **DevTools inspect**.

> **Pitfall:** Swapping padding and margin. If the whole box moved when you wanted the text to move, you used margin; if the text moved when you wanted the box to move, you used padding. Inside/outside is the entire distinction.

How CSS Represents Colour

The problem: "green" is not a colour system

Every example so far has coloured things with words: `red`, `green`, `blue`, `black`, `white`. That works for demos, but reason about its limits: a real design does not want *green*, it wants one exact brand green, distinguishable from a thousand neighbouring greens — and a word vocabulary has, at best, a word per memorable colour. CSS needs a way to name **any** colour precisely. Day 2 closes with three representation systems, in increasing power.

Named colours: words, for quick work

`color: red;`, `color: green;`, `color: blue;` — simple and readable, but **limited**: only practical for **common colours in quick demos**. There is no word for the exact blue of a company logo. Named colours are the vocabulary you start with, not the one you ship with.

Hexadecimal: `#RRGGBB`

The professional default. A hex colour is a **6-character alphanumeric code preceded by `#`**, structured as three pairs:

```
#  3B        59        98
|   |         |         |
|   red       green     blue
prefix  shade     shade     shade

#RRGGBB  →  first 2 chars = red, next 2 = green, last 2 = blue
```

The lecture's example: **`#3B5998` is Facebook blue** — a specific, reproducible, brand-exact colour that no word names. Two honest observations from the session:

- **No human can guess the colour by reading the code alone.** `#3B5998` does not *look* blue on the page of code; the system is for machines and precision, not readability.
- **Code editors compensate**: they display a **colour swatch next to the hex value**, so the developer sees the colour inline while editing. The tooling bridges the readability gap.

And where do developers *get* a hex code for a colour they want? The instructor's recommended picker: **materialui.co/colors** — **click any shade to copy its hex code instantly**, then paste it into your CSS.

The lecture's analogy makes the `RRGGBB` structure intuitive: **paint vendors like Asian Paints and Nerolac mix red, green, and blue dynamically to produce any customer-chosen colour** — you point at a shade on the chart, the machine computes the mixture. A hex code is exactly that: **the**

mix formula, recording how much red, how much green, and how much blue combine into the one colour you chose.

The RGB function: `rgb(red, green, blue)`

The same mixture idea expressed as a function with three numeric arguments, each from **0 to 255**:

```
color: rgb(255, 0, 0);
```

The anchor values from the lecture — learn the corners of the cube and every other colour is a mixture of them:

RGB value	Colour	Reading
`rgb(255, 0, 0)`	Pure red	Red at full, others zero
`rgb(0, 255, 0)`	Pure green	Green at full
`rgb(0, 0, 255)`	Pure blue	Blue at full
`rgb(255, 0, 255)`	**Pink** (red + blue mixed)	Two channels at full
`rgb(255, 255, 255)`	**White**	All channels at full

Read the table as arithmetic: each number says how much of that light to add. Nothing of anything is black (`rgb(0, 0, 0)`); everything at full is white; red and blue together, with green withheld, give the pink/magenta of the fourth row. **Colour pickers can output RGB values directly** — choose the RGB tab instead of hex in the picker and paste the resulting `rgb( … )` into your CSS.

RGBA: RGB plus opacity

`rgba()` extends RGB with a **fourth argument controlling opacity**:

```
color: rgba(59, 89, 152, 0.5);
```

- **0 = fully transparent** (the colour is invisible; what is behind shows through completely)
- **1 = fully opaque** (the ordinary solid colour)

This is a **separate function from `rgb()`** — the trailing `a` (alpha) and the fourth number are the difference. Its home ground is **overlay effects**: a translucent dark layer over a photograph so text above it stays readable, a coloured tint over an image — any time the design wants a colour *and* the thing behind it, blended in a stated proportion.

Background: opacity here attaches to *this colour on this element*, which is subtler than it sounds — `rgba` lets a background be see-through while the text sitting in the same element stays fully solid, because the transparency lives in the background's colour value, not in the element as a whole. That independence is exactly why the colour-level alpha channel exists.

Which format should you use?

The lecture closes the topic by refusing to close it: **there is no strict rule. No restriction** governs which format production code uses; hex and RGB are **both standard**, and in the instructor's words, *"choice is totally yours — there is no restriction."* Pick hex for compact brand colours copied from pickers, RGB when you are thinking in mixtures, RGBA the moment transparency enters the design — the browser treats all three as first-class.

> **Exam:** Named colours = limited, demos only. **Hex** `#RRGGBB`: first pair red, second green, third blue (`#3B5998` = Facebook blue; get codes from **materialui.co/colors**, editors show a swatch). `rgb(r, g, b)`: each 0–255; `255,0,0` red, `0,255,0` green, `0,0,255` blue, all-255 white. `rgba` adds a 4th argument, **opacity 0 (transparent) to 1 (opaque)** — for overlays. No mandated format; choice is yours.

Combined Worked Example: A Styled Profile Card Page

The exercise

Build a small page that has to use *almost everything from the evening* at once: a page-wide base style, a heading that is simultaneously targeted by all four selector types (so specificity decides its colour), a shared class styling several different elements, external CSS linked from a separate file, one inline style overriding that file, font and text properties, a centred image, and a boxed card `div` whose border, padding, margin and fixed size only behave because of `box-sizing: border-box` — finished in hex, RGB and RGBA colours.

Two files. Save them **in the same folder** (the `href` is a relative path), then open `day2-demo.html`:

File 1: `day2-style.css` (external CSS — rules only, no `<style>` tag)

```
/* ═══════════════════════════════════════════════════════
   day2-style.css — EXTERNAL CSS
   No <style> tag in a .css file: selectors and rules directly.
   ═══════════════════════════════════════════════════════ */
```

```css
/* UNIVERSAL selector: one property for every element on the page.
   Preferred over styling body, which can affect backgrounds. */
* {
    font-family: Arial;
    font-size: 16px;
}

/* TAG selector: EVERY h2 on the page shares this style. */
h2 {
    color: rgb(0, 128, 0);
    text-decoration: underline;
}

/* SPECIFICITY DEMO on the main h1 — four rules, one winner.
   Universal says blue, tag says green, class says red,
   ID says yellow.  ID > Class > Tag > Universal, so: yellow. */
* {
    color: rgb(0, 0, 255);
}

h1 {
    color: rgb(0, 128, 0);
    font-size: 32px;          /* not set by the ID rule, so this survives */
}

/* CLASS selector: one rule, reused by h1 + p + span (different tags).
   Only its color loses to the ID rule on the h1; on the p and span
   nothing stronger competes, so they really do turn red. */
.blue-font {
    color: rgb(255, 0, 0);
    font-style: italic;
}

/* ID selector: one specific element only; id must be unique. */
#head1 {
    color: #FFD700;           /* hex gold — the lecture's winning "yellow" */
    text-transform: uppercase;
}

/* Centering the image with CSS — never align=, never <center>.
   img becomes a block; auto left/right margins centre the block. */
#photo {
    display: block;
    margin-left: auto;
    margin-right: auto;
    border-width: 4px;
    border-color: black;
    border-style: solid;
    border-radius: 20px;
}
```

```css
/* The card: a div = one page section, styled as a fixed box.
   box-sizing: border-box keeps it 300x300 INCLUDING padding
   and border; without it the box would grow past 300. */
#card {
    width: 300px;
    height: 300px;
    box-sizing: border-box;
    background-color: #3B5998;      /* hex: Facebook blue */
    color: white;                    /* inherited by children inside */
    border-width: 10px;
    border-color: black;
    border-style: dashed;            /* try solid / dotted / double */
    border-radius: 20px;
    padding: 20px 10px 15px 40px;    /* 4 values: top right bottom left */
    margin-left: 50px;               /* whole box shifts 50px right */
    margin-top: 20px;
}

/* RGBA overlay strip inside the card: black at 50% opacity,
   so the blue card shows through beneath the white text. */
.overlay {
    background-color: rgba(0, 0, 0, 0.5);
    color: rgb(255, 255, 255);
    font-weight: 900;
    text-align: center;
    padding: 10px 20px;              /* 2 values: top/bottom, left/right */
}

/* Small print: font-variant + a line through the old price. */
.smallprint {
    font-variant: small-caps;
    font-size: 14px;
}

.old-price {
    text-decoration: line-through;
}
```

File 2: `day2-demo.html`

```html
<!DOCTYPE html>
<html>
<head>
    <title>WPT Day 2 - CSS Demo</title>
    <!-- EXTERNAL CSS: linked, or style.css would never apply.
         rel="stylesheet"  = this file's relationship to the page
         type="text/css"   = textual content (kept for compatibility)
         href              = relative path, same folder as this file -->
    <link rel="stylesheet" type="text/css" href="day2-style.css">
```

```
    <!-- INTERNAL CSS: same file, <style> inside <head>.
         Beats external on conflict, loses to inline. -->
    <style>
        h2 {
            color: #3B5998;   /* overrides the external h2 green */
        }
    </style>
</head>
<body>

    <!-- This h1 matches ALL FOUR selectors (*, h1, .blue-font, #head1).
         Specificity: ID wins color → hex gold.
         font-size 32px survives from the tag rule (ID never set it),
         font-style italic survives from the class rule (ditto). -->
    <h1 id="head1" class="blue-font">Student Profile</h1>

    <!-- Internal CSS recolored this h2; the external underline,
         being uncontested, still applies. -->
    <h2>Personal Details</h2>

    <!-- Class reused across DIFFERENT tags: both turn red + italic. -->
    <p class="blue-font">Course: PG-DAC</p>
    <span class="blue-font">Institute: CDAC</span>

    <!-- INLINE CSS: style attribute on the tag itself.
         Highest preference of the three types - this p is purple
         even though the universal selector said blue. -->
    <p style="color: rgb(128, 0, 128);">Fees status: Paid</p>

    <!-- Centered by CSS (#photo rule) - no align attribute, no <center>. -->
    <img id="photo" src="photo.jpg" alt="Student photo" width="150" height="150">

    <!-- The card: one div = one section. Everything inside it
         inherits color: white from the #card rule. -->
    <div id="card">
        <h3>Special Offer</h3>
        <p>Grouped elements live inside one div and move as a unit.</p>
        <p>Price: <span class="old-price">Rs. 999</span> Rs. 499</p>
        <div class="overlay">Limited Period Offer</div>
    </div>

    <p class="smallprint">Terms And Conditions Apply</p>

</body>
</html>
```

> **Pitfall:** `photo.jpg` must sit in the same folder (or change the `src` path) — a missing image shows the `alt` text, which is the browser's silent-failure style again, not an error. If the whole page renders unstyled, check the `<link>` tag and the `href` path before touching any rule.

Trace it end to end

1. **The link decides whether any of this exists.** The browser loads `<head>` first, follows `<link href="day2-style.css">`, and loads the external rules; then it reads the internal `<style>` block. Delete the `<link>` line and every external rule — the universal base, the specificity demo, the card — vanishes at once, leaving only the internal `h2` colour and the one inline style. That is "unlinked CSS is never applied", made visible.
2. **The main heading is the specificity ladder, rendered.** The `<h1 id="head1" class="blue-font">` matches all four selector types. `color` is contested by all four, so the ID's hex gold `#FFD700` wins — the lecture's yellow result. But `font-size: 32px` from the *tag* rule and `font-style: italic` from the *class* rule also appear, because neither was contested by the ID rule: **only conflicting properties are overridden**, and the final heading is a merge of three rules, not a victory parade by one.
3. **The class proves its purpose on the next two lines.** `class="blue-font"` on a `<p>` and a `<span>` turns both red and italic from *one* rule in the external file. No ID could do this (IDs are unique); no tag rule could do it (it would take every `p` and `span`). Change `rgb(255, 0, 0)` once and both elements — plus nothing else — change.
4. **The `h2` shows internal beating external.** The external file colours every `h2` `rgb(0, 128, 0)` green; the internal `<style>` block re-colours `h2` to `#3B5998`. Internal > external, so the heading is Facebook blue — while `text-decoration: underline`, set only externally and contested by nobody, still applies. The three-types preference order and the only-conflicts rule, in one element.
5. **The purple paragraph shows inline beating everything.** The universal selector set `color: rgb(0, 0, 255)` on *every* element, this `<p>` included. Its inline `style="color: rgb(128, 0, 128);"` overrides that — inline is the "walk into the house" precision the lecture described, the highest preference of the three types. One element, deliberately dressed differently from the entire page, at the price of a rule nobody can reuse.
6. **The image is centred by CSS alone.** `#photo` sets `display: block` with `margin-left: auto; margin-right: auto;` — no `align` attribute, no deprecated `<center>` tag — and the solid 4px border with `border-radius: 20px` rounds its frame. This is the session's opening question, answered in its prescribed form.
7. **The card is one `div` behaving as a section.** `#card` fixes it at 300×300, paints it `#3B5998`, and sets `color: white` once — and the `<h3>`, both paragraphs and the offer strip inside all

render white without a single rule of their own. `border-style: dashed` is switchable to `solid` / `dotted` / `double` to replay the border demonstration.

8. **The box model's arithmetic is the quiet test.** `padding: 20px 10px 15px 40px` reads clockwise — top 20, right 10, bottom 15, left 40. With `box-sizing: border-box`, the card still measures exactly 300×300 in DevTools: padding and the 10px border are absorbed inward. Comment out the `box-sizing` line and re-measure — the card grows beyond 300 in both dimensions, replaying the lecture's 300×300 → 359×329 demonstration. Open the DevTools box-model panel on `#card` and read content, padding, border, margin (left 50, top 20) straight off the diagram.
9. **Three colour systems coexist without friction.** Hex (`#3B5998` , `#FFD700`), RGB (`rgb(0, 0, 255)` , `rgb(255, 255, 255)`) and RGBA — the `.overlay` strip's `rgba(0, 0, 0, 0.5)` lets the card's blue show through half-strength black while its white text stays solid. One property syntax, one browser, no hierarchy among formats — choice is totally yours.
10. **Text properties finish the page.** `text-transform: uppercase` shouts the heading without the HTML text being typed in capitals; `font-variant: small-caps` dresses the terms line; `line-through` strikes the old price `Rs. 999` ; `font-weight: 900` heavies the offer strip; `Arial` from the universal rule typesets everything — provided, as always, the visitor's system has it.

Quick Revision Checklist

Topic	One-line takeaway
Selector	The address half of `selector { property: value; }` — names *which* elements a rule reaches
Tag selector	`h1 { }` — every element of that kind (100 `h1` s = 100 styled)
ID selector	`#head1 { }` — exactly **one** element; ID unique per page; `id` allowed on any tag
Class selector	`.blue-font { }` — **many** elements, any tags, one rule instead of duplicated blocks
Universal selector	`* { }` — every element; preferred over styling `body` (background side-effects)
The `#` / `.` prefixes	Missing `#` = tag selector that matches nothing; `#` = ID (one), `.` = class (many)

Topic	One-line takeaway
Duplicate IDs	HTML tolerates them, console warns, bugs follow — "you won't get errors, but bugs will follow you"
Specificity	**ID > Class > Tag > Universal**; lecture demo: blue / green / red / **yellow — ID won**
Conflicts only	Specificity settles **only contested properties**; uncontested ones from weaker rules still apply
Internal CSS	`<style>` inside `<head>` of the same file; fine for one page, repeats across files
External CSS	Separate `.css` file (rules directly, **no** `<style>` **tag**); inert until `<link>` ed
The link tag	`<link rel="stylesheet" type="text/css" href="style.css">` in `<head>`; `href` is a relative path
External's payoff	**Modularity + reusability** — one file linked to 50+ pages, one edit updates all
Inline CSS	`style="…"` on the tag; most accurate/highest preference, least maintainable
Types preference	**Inline > Internal > External** on conflict; non-conflicting properties still merge
Image centering	CSS only — `align` attribute not to be used; `<center>` **deprecated in HTML5**
`font-size`	Text size in px (`24px`)
`font-family`	Typeface by name — **must exist on the user's system** or it silently falls back
`font-weight`	Numeric boldness (`900` = heavy); real result limited by the family's weights
`font-style` / `font-variant`	`italic` / `oblique` slant; `small-caps` = capitals with larger first letters (rarely used)
`text-transform`	`uppercase` / `lowercase` / `capitalize` — display case only, HTML text unchanged
`text-decoration`	`underline` (below) / `overline` (above) / `line-through` (through = cut/deleted)
`div`	"Division" — container breaking the page into sections (navbar, footer, slider, sidebar)
`div` defaults	Width **100% of parent**, height **from content** (18.8px → ~59px demo), background **inherited**

Topic	One-line takeaway
Many divs	Distinguish and style individually with **ID** (`#navbar` , `#footer`)
Border	Visible border needs **three**: `border-width` , `border-color` , `border-style`
Border styles	`solid` / `dashed` / `dotted` / `double` ; `border-radius: 20px` rounds corners
Padding	**Inside**: border ↔ content; moves content, not the box
Margin	**Outside**: box ↔ neighbours; moves the whole box (`margin-left: 50px`)
Shorthand	4 values = **top, right, bottom, left (clockwise)**; 2 = top/bottom + left/right; 1 = all equal
`box-sizing: border-box`	Declared size **includes** padding+border — 300×300 stays 300×300 (else grew to 359×329)
Box model	Content → padding → border → margin; diagram visible in **DevTools inspect**
Named colours	`red` , `green` , `blue` — quick demos only, too limited for real colours
Hex	`#RRGGBB` — red, green, blue pairs; `#3B5998` = Facebook blue; pick at materialui.co/colors
RGB	`rgb(r, g, b)` , each 0–255; `255,0,0` red, `0,255,0` green, `0,0,255` blue, all-255 white
RGBA	4th argument = opacity, **0 transparent** → **1 opaque**; separate function, for overlays
Format choice	Hex and RGB both standard — "choice is totally yours, there is no restriction"

Practice Exercises

Work these without looking at the answers. Most are built on the exact confusions the lecture called out — the missing `#` , the grown box, the border that never appeared.

Concepts

1. A page has 40 `<h2>` elements. Requirement A: all 40 must be green. Requirement B: exactly one of them — the third — must also be orange. Name the selector type for each requirement and write both rules.

2. A student writes `head1 { color: red; }` intending to style `<h1 id="head1">`, and nothing changes. Explain precisely what the browser did with that rule instead.
3. The second `<h1>`, first `<p>` and third `<span>` must all be blue; nothing else changes. Why can neither a tag selector nor three ID selectors do this *well*, and what is the correct solution? Write it.
4. True or false, with a reason: "I created `style.css` in my project folder, so my page should now be styled."
5. A classmate proposes styling every inline, "because inline has the highest preference, so it must be the best type of CSS." Dismantle the reasoning in two sentences.

Specificity and the three types

6. An element matches: `* { color: blue; }`, `p { color: green; }`, `.note { color: red; }`, `#main { color: yellow; }`. (a) What colour is it? (b) The ID rule is deleted — now what colour? (c) Instead of deleting anything, the universal rule also sets `font-style: italic` and no other rule mentions `font-style`. What is the element's final style?
7. The same `color` for one element is set in an external file (blue), an internal `<style>` block (green), and an inline `style` attribute (red). Which colour renders, and which type is described by the lecture's "walking into someone's house" analogy?
8. Your site has 60 pages sharing one design. A colour must change everywhere. Compare the work required under internal CSS versus external CSS, and name the two advantages of external CSS that your comparison illustrates.

Fonts, text, divs and borders

9. You declare `font-family: Algerian;` and `font-weight: 900;`. A visitor's machine has Algerian installed but with only regular and bold weights. Describe what the visitor sees and why.
10. Without changing any HTML text, make a heading display as (a) ALL CAPITALS, (b) First Letter Of Each Word capitalised, (c) struck through as if deleted. Name the property and value for each.
11. An unstyled `<div>` containing one line of text is measured in DevTools. Predict its width behaviour, its height behaviour, and its background — then state which single CSS property changes each one.
12. Spot the bug: `div { border-width: 5px; border-color: black; }` renders no border at all. What is missing, why does its absence defeat the other two, and what four values could supply it?

Padding, margin, box model and colour

13. Write out the four sides for each shorthand: (a) `padding: 20px 10px 15px 40px;` (b) `padding: 10px 20px;` (c) `margin: 8px;`
14. A 300×300 box gains `padding: 30px;` and renders noticeably larger than 300×300. (a) Why? (b) What one property restores exactly 300×300, and what shrinks instead to make room? (c) In the lecture's demonstration, what did the 300×300 div grow to *without* that property?
15. A student sets `margin-left: 20px` expecting the text inside a box to move away from the box's left border; instead the whole box moves. Name the property they actually needed and state the inside/outside rule that separates the two.
16. Decode and encode: (a) In `#3B5998`, which pair of characters is the green shade? (b) Write the `rgb()` for pure blue. (c) Write the `rgb()` from the lecture that produces pink, and explain it as a mixture. (d) You need that pink at half transparency over a photo — write the full value and name the function.
17. Where in the browser do you look to confirm a box's content size, padding, border and margin without guessing — and in the lecture's example, what were the content dimensions and the margin?

Practice Answers

1. Requirement A is a **tag selector** — every element of the kind shares the style. Requirement B is an **ID selector** — one specific element singled out (its ID must be unique on the page, and in a real page the third `h2` would carry that `id`):

```
h2 {
    color: green;
}

#third-heading {
    color: orange;
}
```

Note the interplay: on that one heading both rules set `color`, so by **ID > Tag** specificity it renders orange; the other 39 stay green.

2. Without the `#`, CSS reads a bare word as a **tag selector by default**. The rule went looking for `<head1>` elements; none exist, so it matched nothing and was silently ignored — no error, in line with "you won't get errors, but bugs will follow you". Fix: `#head1 { color: red; }`.

3. A tag selector is too broad — it would colour *every* `h1`, `p` and `span`. Three ID selectors work mechanically but duplicate the same `color: blue;` block three times, violating ID uniqueness discipline in spirit and forcing three edits for one change. The class selector exists for exactly this group:

```
<h1 class="blue-font">Second heading</h1>
<p class="blue-font">First paragraph</p>
<span class="blue-font">Third span</span>
```

```
.blue-font {
    color: blue;
}
```

4. False. An external `.css` file is inert until it is **explicitly linked** to the HTML file with `<link rel="stylesheet" type="text/css" href="style.css">` inside `<head>`; otherwise the CSS is not applied at all. (And the file itself must contain rules directly — no `<style>` tag inside a `.css` file.)

5. Preference order answers *which rule wins a conflict*, not *which placement is best to write*. Inline wins conflicts precisely because it is attached to one element — which is also why it is the **least maintainable at scale**: no reuse, no single place to edit. External CSS, the *weakest* in conflicts, is the professional default because of modularity and reusability.

6. (a) **Yellow** — ID is highest in **ID > Class > Tag > Universal**, the lecture's live result. (b) **Red** — with the ID rule gone, the class rule is the strongest remaining. (c) `font-style: italic` **applies together with** the winning colour — specificity overrides **only conflicting properties**; `font-style` was uncontested, so the weaker universal rule still contributes it.

7. Red — the inline style wins: preference is **Inline > Internal > External**. The "walking into someone's house to shoot — most accurate, highest preference" analogy describes **inline** CSS, attached directly to the target element.

8. Under **internal** CSS, the rule is repeated inside every page's `<style>` block, so the change means editing 60 files (and inevitably missing one). Under **external** CSS, all 60 pages link one `style.css`; **a single edit updates all of them**. The two advantages illustrated are exactly the lecture's pair: **modularity** (HTML and CSS in distinct files) and **reusability** (one file serving 50+ pages).

9. The typeface renders — **the font exists on the user's system**, which is the condition for `font-family` to work. The weight, however, renders as the family's **bold**, not a true 900: the **actual visual result depends on how much weight variation the chosen font family supports**, and this family offers only regular and bold. The number requests; the font decides what it can supply.

10. All three are display-only changes via CSS; the HTML text is untouched. (a) `text-transform: uppercase;` (b) `text-transform: capitalize;` (c) `text-decoration: line-through;` — the strikethrough that marks text as cut/deleted.

11. Defaults: **width = 100% of the parent/browser width**; **height = whatever the content needs** (the lecture's demo: ~18.8px for one paragraph, growing to ~59px when an `h3` was added); **background = inherited from `body`** (white), with no colour of its own. Changed respectively by `width`, `height`, and `background-color` — e.g. the lecture's `width: 300px; height: 300px; background-color: green;` (with `color: white;` also restyling all children).

12. `border-style` is missing. Thickness and colour describe a line's properties, but until a **style** declares how the line is drawn there is no line to be thick or coloured — so nothing renders, silently. Supply any of the four demonstrated values: `solid`, `dashed`, `dotted`, or `double`.

13. (a) Four values read **clockwise — top 20px, right 10px, bottom 15px, left 40px**. (b) Two values — **top and bottom 10px, left and right 20px**. (c) One value — **all four sides 8px**.

14. (a) By default, declared `width` / `height` measure the **content only**, and padding stacks outside that measure, so the rendered box grows by the padding on each side. (b) `box-sizing: border-box;` — the declared 300×300 then *includes* padding and border, and the **content area shrinks** to absorb them. (c) In the lecture's demonstration the 300×300 div grew to **359×329** without `border-box`, and stayed **300×300** with it.

15. They needed **padding**. **Padding is internal** spacing — between the element's border and its content, moving the content while the box stays put. **Margin is external** spacing — between the box's outer edge and surrounding elements, moving the whole box, which is what `margin-left` did.

16. (a) The **middle pair,** `59` — `#RRGGBB` is red pair, green pair, blue pair in order. (b) `rgb(0, 0, 255)`. (c) `rgb(255, 0, 255)` — red at full plus blue at full with green at zero; the lecture's red + blue mixture producing pink. (d) `rgba(255, 0, 255, 0.5)` using **RGBA**, the separate function whose fourth argument is opacity — 0 fully transparent to 1 fully opaque.

17. The **DevTools inspect panel's box-model diagram**, which displays content dimensions, padding, border thickness and margin for the selected element — for debugging layout without guessing. The lecture's example: **content 240×270px**, padding-left 20, padding-top 10, padding-right 20, padding-bottom 0, border ≈ 9.6px, and **margin 0**.

Coverage: WPT Day 2, evening session (22 Oct 2026) — the four CSS selectors (tag, ID, class, universal) and their prefixes, specificity (ID > Class > Tag > Universal, conflicts only) with the blue/green/red/yellow demonstration, the three types of CSS (internal, external with the `<link>` tag's three attributes, inline) and their preference order, image centering via CSS over `align` / `<center>`, the seven font and text properties, the `div` tag and its defaults, borders and `border-radius`, padding vs margin with shorthand order and `box-sizing: border-box`, the DevTools box model, and colour as named/hex/RGB/RGBA — every concept rebuilt from first principles, with the lecture's own examples, analogies, and live demonstrations preserved.